

ATIVIDADE:: CONCEITOS SOBRE TESTES (1)

1. Recapitulação de projetos anteriores e enumeração dos testes realizados

- **Enumeração de testes em projetos anteriores:** Nos meus projetos anteriores, eu costumava fazer testes mais básicos e práticos, como testar manualmente as telas clicando nos botões para ver se a interface respondia, preencher formulários com dados certos e errados para ver se validava, e rodar o programa inteiro no final para ver se o resultado batia com o que eu esperava.

2. Descrição dos testes realizados utilizando os conceitos da disciplina

- **Estado do Programa:** É basicamente como as variáveis estão guardadas na memória em um momento exato da execução. *Exemplo prático:* Quando eu fazia uma aplicação de cadastro, checava se a variável do nome realmente guardava o texto que eu digitei antes de salvar.
- **Erro (Error):** É quando o programa entra num estado errado ou inesperado enquanto está rodando. *Exemplo prático:* Acontecer um estouro de memória ou uma variável vir nula sem eu esperar.
- **Falha (Failure):** É o problema que o usuário consegue ver de fato, quando o programa se comporta de um jeito diferente do esperado por causa daquele erro interno. *Exemplo prático:* O aplicativo fechar sozinho (crashar) ou mostrar uma tela branca na hora que o usuário clica em salvar.
- **Defeito (Fault):** É a falha que a gente comete no código-fonte ou na lógica (o famoso bug). *Exemplo prático:* Eu ter esquecido de colocar um sinal de igual numa condição if ou ter errado uma fórmula matemática no código.
- **Caso de Teste:** É o conjunto de entradas que eu dou para o sistema junto com o resultado que eu espero receber de volta para saber se passou. *Exemplo prático:* Passar a nota 7 e a nota 5 para uma função de média e esperar que o resultado retornado seja 6.

3. Referências na literatura acadêmica envolvendo erro, falha, defeito e bug

- **Referência principal (Livro adotado na disciplina):**
 - **Título:** *Introdução ao Teste de Software* (Autor: Marcio Delamaro e colaboradores, Editora Campus/Elsevier).
 - **Relação com os termos:** No primeiro capítulo, o livro explica bem a diferença entre esses conceitos fundamentais da engenharia de software. Ele mostra que o programador insere um **defeito** (*fault*) no código, que durante a execução gera um **erro** (*error*), que por sua vez

acaba se manifestando como uma **falha** (*failure*) visível para quem está usando o sistema.

ATIVIDADE:: MOTIVAÇÕES

1. Pesquisa de casos notáveis onde a falta de testes foi apontada como razão principal para problemas graves

- **Relato do caso:** Um dos casos mais emblemáticos e trágicos da história da computação foi o do **Therac-25** (entre 1985 e 1987), uma máquina de radioterapia controlada por computador. Devido a falhas de concorrência e à falta de testes de software adequados em sistemas embarcados e de segurança crítica, o equipamento aplicou doses massivas e letais de radiação em pacientes. O problema ocorria apenas em condições específicas de digitação rápida por parte do operador, revelando que a ausência de testes rigorosos de cenários de uso e falhas em tempo de execução pode custar vidas humanas.
- **Referências:**
 - Leveson, N. G., & Turner, C. S. (1993). An investigation of the Therac-25 accidents. *IEEE Computer*, 26(7), 18-41.

2. Relatos de experiência apontando sucesso ou melhoria significativa em projetos atribuídos à adoção de boas práticas de testes

- **Relato de experiência:** Em projetos que adotam metodologias ágeis combinadas com práticas de *Test-Driven Development* (TDD) e automação de testes contínua (como relatado em diversos estudos de caso de engenharia de software em grandes empresas e equipes de código aberto), observa-se uma reviravolta impressionante na qualidade do produto. O sucesso costuma ser medido pela queda drástica de defeitos encontrados em produção (muitas vezes reduzidos em mais de 40% a 80%), diminuição drástica no tempo gasto com depuração manual (*debugging*) e aumento da confiança dos desenvolvedores para refatorar o código e lançar novas versões rapidamente sem medo de quebrar funcionalidades antigas.

ATIVIDADE:: ATORES

1. Pesquisa de vagas abertas envolvendo testes de software

- **Relação de habilidades e tecnologias requeridas (por nível):**

- **Júnior (ex.: Analista de Qualidade / Testes Júnior):** Foco em conceitos fundamentais do ciclo de desenvolvimento de software (SDLC), execução de testes funcionais e manuais, criação básica de cenários de teste, documentação de defeitos em ferramentas como Jira e facilidade de comunicação para reportar falhas à equipe de desenvolvimento.
- **Pleno (ex.: Analista de Automação de Testes Pleno):** Requer sólidos conhecimentos em lógica de programação e linguagens de script (como Python ou Java), domínio de frameworks de automação (Selenium, Playwright ou Cypress), testes de API (Postman/REST), e integração dos testes em pipelines de CI/CD (GitHub Actions, Jenkins).
- **Sênior (ex.: Senior Software Test / QA Engineer):** Exigência de experiência avançada na arquitetura de estratégias de teste completas, certificações reconhecidas (como a ISTQB), liderança técnica e mentoria de profissionais juniores, além de proficiência em testes de carga/desempenho e integração com ferramentas modernas (incluindo o uso de IA generativa para otimização de testes).
- **Referências utilizadas:** Plataformas globais e locais de recrutamento e vagas de tecnologia, como *LinkedIn Jobs*, *Indeed* e portais de carreiras corporativas (Siemens, IgniteTech).

2. Resumo sobre a abordagem de testes em grandes empresas e perspectivas

- **Como grandes empresas têm abordado a área de testes?**
 - Grandes corporações tratam a qualidade de software como uma responsabilidade compartilhada e integrada desde o início do ciclo de desenvolvimento (cultura de *Shift-Left*). O processo é altamente automatizado, rodando baterias massivas de testes integradas diretamente aos pipelines de entrega contínua (CI/CD) para garantir validações rápidas a cada alteração de código.
- **O que se espera de um profissional nesta área?**
 - Espera-se que vá muito além do teste manual repetitivo. O profissional moderno precisa dominar a criação de códigos para automação, entender de arquitetura de sistemas, saber analisar dados de telemetria e falhas em produção, e atuar de forma colaborativa e estratégica junto aos desenvolvedores e gerentes de produto.

- **Quais as perspectivas futuras para esses papéis e profissionais?**
 - A perspectiva é de forte valorização e evolução técnica, impulsionada principalmente pela automação inteligente e pela integração de ferramentas de Inteligência Artificial para a geração e otimização de cenários de teste. O papel do testador migra cada vez mais para o de um engenheiro focado em estratégia de qualidade, confiabilidade e governança de dados.

ATIVIDADE:: TIPOS DE TESTE

1. Tipos de teste, técnicas e ferramentas

- **Testes de Unidade:** Focados em validar componentes ou funções individuais de forma isolada.
 - **Técnicas:** Análise de valor limite, partição de equivalência e teste estrutural (cobertura de código/caminhos).
 - **Ferramentas:** JUnit, NUnit, Jest.
 - **Referências:** ISTQB (International Software Testing Qualifications Board) Foundation Level Syllabus; livros de engenharia de software de Ian Sommerville.
- **Testes de Integração:** Focados em verificar a comunicação e a interface entre diferentes módulos ou subsistemas integrados.
 - **Técnicas:** Integração incremental (ascendente/descendente), testes de stubs e drivers.
 - **Ferramentas:** Testcontainers, Spring Test, Postman (para integração de APIs).
 - **Referências:** Documentação oficial do JUnit e guias de integração contínua corporativa.
- **Testes de Sistema / Aceitação:** Validação do sistema completo frente aos requisitos funcionais especificados.
 - **Técnicas:** Desenvolvimento Orientado a Comportamento (BDD), matriz de rastreabilidade de requisitos.
 - **Ferramentas:** Selenium, Cypress, Cucumber.

- **Referências:** Manuais de metodologias ágeis e padrões de teste ágil (*Agile Testing* de Lisa Crispin e Janet Gregory).
- **Testes de Desempenho / Carga:** Avaliação do comportamento, tempo de resposta e estabilidade do sistema sob alta demanda.
 - **Técnicas:** Testes de estresse, teste de pico (*spike testing*) e testes de longa duração (*endurance*).
 - **Ferramentas:** Apache JMeter, k6, Gatling.
 - **Referências:** Documentação do Apache JMeter e padrões de engenharia de performance web.

2. Exemplos de artefatos produzidos

- **Casos de Teste Documentados:** Planilhas ou arquivos em ferramentas de gestão contendo o identificador, pré-condições, passos para execução e resultados esperados.
- **Relatórios de Cobertura de Código (Code Coverage):** Arquivos gerados por ferramentas como JaCoCo ou Istanbul que mostram visualmente quais linhas, ramificações e funções do código foram ou não exercitadas pelos testes de unidade.
- **Matriz de Rastreabilidade:** Uma tabela que mapeia os requisitos do cliente diretamente aos casos de teste criados, garantindo que tudo o que foi pedido está sendo testado.
- **Scripts de Automação:** Arquivos de código-fonte (por exemplo, em JavaScript/TypeScript para o Cypress ou arquivos .feature em Gherkin para o Cucumber) que automatizam as interações com a aplicação.

3. Exemplos de relatórios de execução para três tipos de teste

- **Relatório de Teste de Unidade:**
 - *Exemplo prático:* Um relatório gerado em formato XML ou visualizado no terminal pelo JUnit/Surefire que exibe: [INFO] Running com.exemplo.CalculadoraTest, seguido por [INFO] Tests run: 15, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.452 s, indicando sucesso absoluto na execução da suíte unitária.
- **Relatório de Teste de Integração (API):**
 - *Exemplo prático:* O sumário de execução gerado pelo Newman (CLI do Postman) ou por relatórios em HTML gerados em pipelines de CI, contendo métricas como: total de requisições enviadas (ex.: 50

endpoints testados), tempo médio de resposta (ex.: 120ms) e o status das asserções (ex.: *Assertions passed: 48, failed: 2*).

- **Relatório de Teste de Desempenho / Carga:**
 - *Exemplo prático:* Um painel gerado pelo Apache JMeter ou k6 detalhando o resumo do teste de carga: número de usuários virtuais concorrentes simulados (ex.: 500 usuários), taxa média de requisições por segundo (RPS), percentil 95 ou 99 do tempo de resposta (ex.: $p(95)=450ms$) e a taxa total de erros de requisição (ex.: 0.02% de falhas).

ATIVIDADE:: HISTÓRICO E EVOLUÇÃO

- **Quais foram as primeiras iniciativas para a criação de testes de software?**
 - As primeiras iniciativas surgiram junto com a própria engenharia de software, quando os sistemas começaram a crescer em complexidade e os erros (*bugs*) deixaram de ser apenas anedotas físicas (como a famosa mariposa encontrada no computador Mark II por Grace Hopper em 1947) e passaram a causar prejuízos operacionais e financeiros. Historicamente, os testes nasceram de uma abordagem puramente intuitiva e corretiva (*debugging* após o erro acontecer) e evoluíram gradualmente para métodos sistemáticos de Verificação e Validação (V&V), formalizados por teóricos e padrões industriais nas décadas de 1970 e 1980.
- **Como são feitos testes no Google?**
 - No Google, os testes são amplamente descentralizados e fazem parte da rotina de todo desenvolvedor, apoiados por uma cultura de forte automação e integração contínua. A empresa utiliza uma abordagem conhecida como *Shift-Left*, onde a qualidade é validada o mais cedo possível. Eles contam com diferentes camadas de teste (unitários, de integração e de sistema) rodando de forma automatizada em grandes fazendas de servidores a cada alteração de código, além de contarem com engenheiros especializados em ferramentas de teste (*Software Engineers in Test*) que criam frameworks internos para garantir a escalabilidade e a confiabilidade de seus produtos.
- **Cite um autor de referência sobre o tema "Testes"?**

- **Marcio Delamaro** (autor nacional da obra de referência *Introdução ao Teste de Software*) ou **Glenford Myers** (autor clássico internacional do livro *The Art of Software Testing*).
- **O que deve haver em um plano de testes?**
 - Um plano de testes bem estruturado deve conter, no mínimo:
 1. **Escopo:** O que será testado (funcionalidades, módulos) e o que ficará de fora.
 2. **Estratégia e Metodologia:** Os tipos de testes que serão aplicados (unidade, integração, desempenho, etc.) e as ferramentas utilizadas.
 3. **Critérios de Entrada e Saída:** Condições necessárias para iniciar os testes e os critérios que definem quando eles podem ser encerrados (ex.: atingir 90% de cobertura de código).
 4. **Recursos e Ambiente:** A infraestrutura, ferramentas, hardwares e a equipe alocada.
 5. **Cronograma:** Prazos e marcos para a execução das etapas de teste.
 6. **Riscos e Mitigações:** Possíveis imprevistos (como indisponibilidade de ambiente) e os planos para contorná-los.

